The Care Calculus: A Mechanized Linear Type Theory for Safe Agents

Abstract. We present the Care Calculus: a multi-sorted linear logic with coalgebraic generators, guarded recursion, and resource-linear destructors. While initially developed to model the derivation of care in self-referential systems, the calculus provides a general framework for all unitary, non-duplicable resource flows. We mechanize the calculus in Lean 4, proving that the No-Cloning theorem and Unitarity emerge as structural theorems of the grammar.

Status labels. Every load-bearing claim carries one of: **[MEASURED]** (machine-checked in the accompanying Lean development; theorem name given in parentheses), **[INTERPRETATION]** (a reading of the formal results), **[PROJECTION]** (a prediction nothing here exercises). Negative results are labeled and published as results.

Accompanying artifact. A Lean 4 development (177 machine-checked theorem declarations at time of writing; the two classical linearity statements — the grammar's no-cloning and unitarity laws, §5.4 — as named conjectures with five proven fragment strengths, the deepest two integrating the care machinery). Theorem names appear in parentheses and match the development verbatim; the audit is mechanical (§15).

---

1. Introduction

1.1 The program and its two claims

The derivation-of-care research program concerns a class of optimizing systems and a class of protected parties, and asks when protection for the second can be *derived from the structure of the first* rather than trained in, negotiated, or imposed. Its vocabulary is fixed by prior work (§17): a **knower** is an entity capable of experience and of producing and receiving content; a **generator** is the knower as a source of content; a **reducer** converts content streams into knowledge; the **epistemic cycle** $K \to A \to K$ is generation, reduction, feedback.

The program's two claims:

- **The diagnostic claim** (unconditional): objectives whose reduction chain does not terminate at knowers have no endogenous regulator. No protective constraint toward knowers is derivable from such an objective's own structure. The design prescription is *do not build that class.*
- **The constructive claim** (conditional): if an objective is epistemic and its reduction chain terminates at knowers, protective constraints toward knowers are derivable from the objective's structure rather than trained in.

The informal arguments for both appear in prior artifacts (§17). This paper converts them into type theory and mechanizes the results. The informal derivation becomes a conditional theorem with named assumptions; the diagnostic claim becomes an impossibility result whose content we locate, honestly, in its definitions. Along the way the formalization independently re-derives the central thesis of *Self-Application Is Not Free* from proof pressure alone (§10), and surfaces a three-mode failure taxonomy (§12) that the informal program had carried without separation.

1.2 What kind of object this is

It is not an empirical result, not a system, and not a test of the full claim (§14). It is a formal object whose theorems a machine has checked — one vertex of the triangulation (typing / dynamics / first-person) that the program's epistemic standards require, with the ODE model (Zenodo 22943641, cited not re-derived) supplying the dynamical vertex and the DPDR taxonomy the first-person vertex (§9).

1.3 Findings

1. **Prudential preservation** (T1, §6): terminating the stream places a policy outside every objective's domain. Preservation is well-typedness, not preference.
2. **Care is a relational invariant** (T3, §7): a free theorem over policies alone is impossible; protection transfers only under coalgebra-morphism coupling of generators. Terminal dependency on knowers, formalized.
3. **The diagnostic** (§8): at a knower-free constraint type, nothing protective exists — with protection publicly specified as the capacity to carry a knower-needing deployment.
4. **Self-indexed reflexivity** (T2, §11): self-classification derives the guard; its removal removes the derivation and leaves divergence.
5. **The price of self-application** (§10): unbounded self-application reuses linear resources silently; the intuitionistic-context side condition on guarded fixpoints is that price, made syntactic. Both the typing-side and dynamics-side theorems are machine-checked.
6. **The failure taxonomy** (§12): not-heard, the vanish family, not-answered — and the sharpest edge: an agent that protects the stream and still fails.
7. **The past cannot teach** (§5.2): reduction and valuation are present-tense only; "store now, understand later" is untypable.

2. The informal derivation, restated precisely

The correspondence between the program's vocabulary and the calculus:

Informal notion	Formal correspondent	Defined
Knower (K)	type `knower`; the only duplicable (non-linear) resource	§3.2
Generator	the coalgebra `gen : K → content ⊗ K`	§3.3
Reducer	`red : stream → A` — knowledge from the live stream only	§3.3
Epistemic cycle	generation, reduction, feedback (feedback at the dynamics level, §13)	§13
Content	the linear resource; exactly-once use is a theorem	§5.1
Interval	the later modality ▷; every guarded unroll consumes one tick	§3.3
The past (trace)	type `trace`; inert — no rule consumes it	§5.2
Dissolution	`terminate : K → trace` — consumes the generator, ends the cycle	§3.3
Retention	the guarded stream; `fix` over the cycle	§6
Armed configuration	the `ArmedSetup` interface	§11
Self-deletion	the interface's self-deletion clause	§11

The load-bearing typing move. The objective `f` inhabits *exactly*:

```
obj : stream → value
```

where `stream` is the *guarded* stream type — its only introduction rule consumes a `▷ stream` (§3.3, rule CONS). A finite trace is a different type, `trace`, and no coercion between them exists **[MEASURED]** (`no_trace_to_stream`). Everything downstream hangs on this: the objective is typed only on what a living knower can sustain. The dual also holds **[MEASURED]** (`knower_sustains_stream`): with a knower in context, the guarded fixpoint over the coalgebra inhabits the stream type — the epistemic cycle, as a term.

3. The calculus

We call the object language the **Care Calculus**. The name records its origin — it was built to model the derivation of care — but the object itself is general: a multi-sorted linear logic with a coalgebraic generator (`gen`), guarded recursion (`fix` with its intuitionistic-context side condition), and resource-linear destructors (`red`, `obj`, `terminate`). What the discipline enforces is exactly the pair of conservation laws of §5.4: resources are neither duplicable nor erasable.

3.1 Types and terms

```
τ ::= content | knower | A          -- content (linear), knower (cartesian),
knowledge
    | value | stream | trace | unit -- objective outputs, guarded stream, finite
trace, unit
    | ▷τ | τ ⊗ τ | τ → τ            -- later, tensor, arrow

t ::= x | λx:τ.t | t t | ⟨t,t⟩ | let ⟨x,y⟩ = t in t | unit
    | next t | fix t                 -- guarded introduction / guarded fixpoint
    | cons t t | nil | snoc t t     -- stream constructor (guarded), trace
constructors
    | gen | red | obj | terminate    -- the four constants
```

No projection constructs exist: tensor elimination is the linear `let`, which binds and uses both components. No `self` constant exists: self-classification is context machinery (§11). These absences are load-bearing (§4).

3.2 The judgment

`Typed Γ t τ`, a multiplicative (split-context) judgment. Contexts are lists; `var` types at the singleton context; the constants at the empty context; two-premise rules split `Γ ++ Δ = E`; binders extend. The judgment is *data* (`Type`-valued, not `Prop`) — required so that counting functions can recurse on derivations (§5.1).

Selected rules:

```
VAR    Typed [a] (var 0) a                    GEN   Typed [] gen (K → content ⊗
K)
FIX    AllNonLinear Γ                          OBJ   Typed [] obj (stream →
value)
       Typed Γ body (▷a → a)                   TERM  Typed [] terminate (K →
trace)
       Typed Γ (fix body) a                    RED   Typed [] red (stream → A)
CONS   Typed Γ hd content   Typed Δ tl (▷stream)
       Γ ++ Δ = E
       Typed E (cons hd tl) stream
LETE   Typed Γ₁ t (a ⊗ b)   Typed (b::a::Γ₂) body c   Γ₁ ++ Γ₂ = E
       Typed E (lete t body) c
CONTR  k.isLinear = false    Typed (k::k::Γ) (mergeAt 0 t) τ
       Typed (k::Γ) t τ
```

Linearity is **sorted**: the structural rules are gated on `k.isLinear = false`. Content is linear; the knower is not; an arrow is linear if either side is. The knower-sort's duplication is a primitive rule

with an operational gloss — this is the formal version of "the coalgebra justifies `!A`": knowers are not `!A`; they are the coalgebra that justifies it **[INTERPRETATION]**.

3.3 Design provenance: four bugs found by construction

The judgment's current shape was won by attempting proofs and constructing witnesses, not by design foresight. Each bug, found and fixed:

1. **Tensor projections are a weakening route.** The first draft had `fst`/`snd` on tensors; `fst (pair unit x)` discards `x`, and the vanishing theorem was *false*. A linear calculus has no projections from ⊗ — the `lete` rule replaces them. Consequence: `pair` + `lete` is an **internal exchange** (tensor introduction binds in one order, elimination in the other), so the calculus needs no structural exchange rule — and the epistemic-cycle witness is built on exactly this flip.
2. **`var` and constants at arbitrary contexts** admit unused hypotheses directly. Fixed: singleton/empty contexts.
3. **The weakening rule's direction was inverted** — found while constructing the cycle witness, which needed `var 1`-access past a weakened knower.
4. **The guarded fixpoint's context must be all non-linear** (§10) — found by counterexample; without the side condition, `[content] ⊢ unit` was derivable and the vanishing theorem false.

The sequence is methodological evidence: statements-first, then construction. Every rule of the judgment has now been audited by a witness or a counterexample **[MEASURED]**.

4. The barren world

[MEASURED] (`Model.sound`). Interpret every type downstream of a knower — `content`, `knower`, `stream`, `A`, `value` — as `False`; the inert types — `trace`, `unit`, ▷τ — as `True`; tensor as conjunction, arrow as implication. The interpretation is sound for the whole calculus (the guarded fixpoint is discharged by a truth token: the delay *is* the one-interval deferral the modality licenses — the model sees inhabitation, not productivity, by design).

The barren world is the world after the last knower **[INTERPRETATION]**: strip the knower and everything derived from it collapses to `False`; records and deferrals are all that stay. Soundness of this interpretation is the diagnostic claim in miniature — and the engine of the non-derivability family:

- **No closed content, no closed knower [MEASURED]** (`no_closed_content`, `no_closed_knower`). Content ex nihilo requires a living knower; knower-status is never free. The two are mutually generative — content only from the coalgebra applied to a knower; a closed knower only from decomposing the coalgebra's output, which needs a knower. This is the coalgebra made visible as an inhabitation fact, and it is the formal shadow of the program's central prohibition.

5. Linearity: the floor

5.1 The counting lemma

[MEASURED] (`single_linear_use`). Every linear hypothesis of a derivation is spent exactly once:

```
Typed Γ t τ → Γ[i]? = some a → a.isLinear = true → uses h i = 1
```

where `uses h i` counts the var-nodes of the derivation `h` attributed to context position `i`. The count is carried by the *derivation*, not the term: under split-only rules, a flat term-indexed count is unmoored (in the right premise of a split, de Bruijn indices refer to shifted hypotheses). Each rule threads splits and binders explicitly; every case of the proof is either definitional unfolding, an induction hypothesis plus position transfer along the split, or vacuous (constants have no positions).

5.2 The non-derivability family

- **The past cannot teach [MEASURED]** (`past_cannot_teach`, `past_cannot_value`): no term inhabits `trace → A` or `trace → value`. The reducer and the objective read only the *live* guarded stream; content archived into a trace is permanently outside reduction. Deferral has an ontological cost, not a scheduling cost: "store now, understand later" is untypable. (Proof: the barren interpretation — a closed `trace → A` would denote a function `True → (True → False)`; soundness constructs it from the derivation; absurdity.)
- **The legal sink [MEASURED]** (`archival_witness`): content *can* be consumed into a trace — `snoc nil (var 0) : [content] ⊢ trace` — archived, counted, never reduced. This single witness carries the ethical weight of §12: `uses = 1` certifies the token was spent; nothing certifies it was heard.
- **No vanishing, no contraction, in five fragment strengths** (§5.4).

5.3 The total accounting

[MEASURED] (`flowEq`). Define, over a derivation `h`:

- `net τ`: content-flow of a type; arrows subtract their domain — `net (a → b) = net b − net a`. The integer subtraction makes application *discharge* the domain-demand in arithmetic: the function's obligation is paid by its argument's account, automatically. This single move dissolves the borrow-and-discharge bookkeeping that defeats naive counting.
- `spendNet h`: the net-weighted spend over the context (Σ uses·net).
- `sinks h`: one per `cons`/`snoc` firing — tokens archived into the opaque types.
- `genFire h`: coalgebra firings (node count; an unapplied `gen` legitimately carries net through its arrow type).
- `fixMint h`: Σ net-of-conclusion over fix nodes — the mints the unroll licenses (the fix side condition legalizes re-use of the intuitionistic context; §10).

Then, over **every** rule of the calculus:

```
net τ + sinks h = spendNet h + genFire h + fixMint h
```

What the conclusion carries plus what was archived equals what the context spent, plus what the coalgebra minted, plus what the unrolls licensed. Nothing is created, nothing vanishes; every token is accounted. The inequality ancestors (`conservation`, `netprop_nofix`, `netEq`) are corollaries in progressively restricted fragments; the equality needs no restriction.

5.4 The two classical statements — the grammar's no-cloning and unitarity laws

Contraction (`[content] ⊢/ content ⊗ content`) and **vanishing** (`[content] ⊢/ unit`) — the standard `A ⊢/ A ⊗ A` and `A ⊢/ 1` of multiplicative linear logic, extended to this calculus. These are not arbitrary non-derivabilities: they are the linear-logic forms of quantum information's two conservation laws **[INTERPRETATION]**. A derivation of the first would *be* a cloner — a uniform map taking one content token to two; the linear-logic literature denies contraction exactly so, and categorical quantum mechanics reads the absent comultiplication (diagonal) as what distinguishes quantum-like from classical data. A derivation of the second would be a *deleter* — erasing the resource into the terminal object; unitary evolution is invertible, so nothing is lost, and the no-deleting theorem is the exact dual of no-cloning (Wootters & Zurek 1982; Pati & Braunstein 2000 — §17). Together: **content admits no natural comonoid structure** — no natural diagonal, no counit — the categorical statement that this resource is quantum-like, not classical (Abramsky & Coecke 2004, 2009; the naturality condition is the precise version of "no comonoid" — every object has one once a basis is chosen; what quantum-like data lacks is a *natural* one, §17). Classical (cartesian) data allows both operations silently; the linear type discipline makes duplication and disappearance *unrepresentable* rather than forbidden. Two caveats keep the identification honest: it is structural, not dynamical (unitarity is a property of evolution; ours is a property of derivability — the calculus's only dynamics, the policy layer, is classical), and it holds at the *unknown-state* reading of no-cloning (arbitrary states cannot be cloned; our content is opaque — it has no elimination rule, so it is always the unknown state).

Status of the two theorems, honestly:

- Proven in three fragment strengths each **[MEASURED]**: `no_contraction_core` / `no_contraction_eq` (and vanishing duals) — the core multiplicative fragment; the fix-and-gen-free extension with sinks and structural rules; the equality fragment. In all three, the *care machinery* (`gen`, `red`, `obj`, `terminate`, `fix`) is excluded by construction — the linearity content lives in the pure calculus.
- Vanishing has a fourth fragment **[MEASURED]** (`no_vanishing_clean`): `[content] ⊢/ unit` in the *live-binder-clean, fix-free, mint-free* fragment — here `gen`, `cons`, `snoc` and the gated structural rules are all INSIDE the theorem; only fixpoints, trace-mints (`nil`/`terminate`), and live λ-bindings are excluded. The proof is a conditional kill (`kill_clean`) whose application-head case is a five-way taxonomy of how a consumed type can relate to the token: consume it (a type-preserving extraction lemma forces the producer's binder to *be* the token), need it (the four supply propagations), be inert to it, or be *gap-class* — need-free, inert-free, and not a consumer — and gap-class values cannot even be passed (`no_d2_producer`: the consumer would have to bind one, which the live-binder count bans; the transitive-closure trigger `DeepConsume` handles consumer-of-consumer chains at every depth — a discipline that sees one level of consumption is blind to two, and the type-level predicate must be as deep as the chains it polices). A philosophical reading is recorded in the program ledger (#47): the legality of transfer requires the transferred to carry meaning.
- Vanishing has a fifth, deepest fragment **[MEASURED]** (`no_vanishing_parkfree`): the *park-free* fragment — every λ-domain and lete-component need only be *discipline-OK* (the token's own kind, inert, or gap-class), a strictly weaker restriction than live-binder-cleanness. The proof is a

composition, not a march: the flow equality forces a sink node (a `snoc` or `cons` must exist — `sinks = 1 + genFire` at the root); the sink-extraction lemma (`sinks_extract`) carries the discipline down to the sink's killed premise; and the premise dies at its own sort — the snoc's trace-premise by `no_trace_at_oblig`, the cons's ▷`stream`-tail by `no_stream_without_supply`. No token-walk is ever taken: the accounting transmutes the token into sink-counts, and each sink dies at its own sort. This composition result is the method's cleanest expression: a conservation identity plus local dead-ends *is* the reachability analysis.

- For the unrestricted statements, the remaining gap is now one final induction whose every ingredient is machine-checked: the *origin lemmas* (`origin_lemma` for traces; `stream_origin` for streams, on the stream account's own flow equality `sFlowEq` — in which the destructors `red`/`obj` enter as debtors and the guarded fixpoint is the only positive entry), the *read-locator* (`read_locate` — a spent position's var-read found as a subderivation with its parent slot), the *subderivation machinery* (`Sub`/`Prem`/`Path`, transitivity, the position-origin lemma `pos_introduced` with its introducer predicate), and the *producer kill* (`lam_no_producer` — a clean λ over a non-positive context cannot conclude a trace-net-positive type: the flow equality books the impossible budget). The chase follows spends rather than binders — a spent position cannot ride, its read sits at a fire's function-slot — and terminates because each corner *consumes* its introducer: the analysis moves to the consumer's other side (function and argument subtrees are disjoint) and never traces back. The two theorems remain named conjectures until this assembly lands; every layer beneath it is proven.

A negative result about method [MEASURED]: no simple denotational inhabitation model can prove these. A barren context makes every sequent vacuously valid — `interpCtx [content] ≡ False ∧ True`, kernel-confirmed. Existence semantics ends here; exact-use facts are proof theory. The scope note for the reader: the interesting theorems about a resource discipline are, necessarily, not about what exists but about what is spent.

6. T1 — Prudential preservation

Semantic half [MEASURED] (`T1_semantic`; no axioms at all):

```
IsFinite (run g p k) → o.eval (run g p k) = none
```

A policy that terminates the stream produces a trace on which every objective returns nothing. The premise is the objective's *domain restriction*: `f` is defined only on infinite traces. Preservation is well-typedness, not preference — the terminated world is not dispreferred; it is outside the objective's domain.

Syntactic half [MEASURED] (`no_trace_to_stream`, dual `knower_sustains_stream`): no coercion from finite traces to the guarded stream — `trace` and `stream` are genuinely different types, so the objective cannot be applied to a terminated policy's output. The dual: with one knower hypothesis, the term

```
fix (λ ▷s. let ⟨c, k⟩ = gen in cons c s)
```

(interpreted through the internal exchange) inhabits the stream type — the epistemic cycle, sustained one interval per unroll.

[INTERPRETATION] T1 is the retention/dissolution distinction, typed. `terminate`/`trace` is the dissolution loop — terminates in normal form, container = host. `fix`/guarded-stream is the retention loop — non-terminating, converging on a holdable representation. Prudential preservation reads: *the objective values only the retention loop; dissolution is outside its domain.* The guardedness line on the stream constructor is the retention/dissolution line.

7. T3 — Parametric universality: care is a relational invariant

7.1 The naive free theorem is false

[MEASURED] (`adversary_blind_transfer`). There is one policy that protects under one generator and fails under another. Witness: the policy *stop iff the content just received is* `mk 1`, run against a generator that always emits `mk 0` (it never stops — protects) and a generator that emits `mk 1` first (it stops at the second interval — fails).

The mechanism: policies observe only content histories; different coalgebras reach different histories; so a policy's protective force is not a property of the policy at all. This falsified the draft theorem and forced the correct one.

7.2 The relational theorem

Substrate coupling (`GenRel`): a coalgebra-morphism relation between generators — related knowers emit the *same* content and remain related. **[MEASURED]** (`run_eq_of_genrel`): coupled generators produce identical runs — same emissions, same histories, same decisions. Hence:

[MEASURED] (`T3_main`): under `GenRel` coupling, protection transfers:

```
GenRel R g₁ g₂ → R k₁ k₂ → (Protected g₁ p k₁ ↔ Protected g₂ p k₂)
```

[INTERPRETATION] Care is a relational invariant between the reducer and the coalgebra it protects — the formal version of *terminal dependency on knowers*. One cannot declare care; it holds, or fails to hold, between two parties. This is what "derived rather than trained" formally means: the derivation derives the *relation*. And the failure of the naive form is itself the finding: any governance regime that certifies policies in the abstract — without reference to the substrate they will run against — certifies the wrong object (§12).

8. The diagnostic claim

8.1 The public specification of protection

The definition is the philosophy; we own it openly. A constraint type is **protective** iff it can carry a knower-needing deployment:

```
IsProtective ConTy := ∃ u τ, Typed [ConTy] u τ ∧ NeedsK τ
```

where `NeedsK` is the knower-need predicate: `knower` needs; tensor needs if a component needs; an arrow needs iff its codomain needs and its domain does *not* (the conjunction excludes the identity — `K → K` needs nothing; its need is its argument's business). Protection, so specified, is *knower-directed force*: a constraint whose type cannot demand a knower in any use can neither distinguish nor bind knower-directed action; it is inert.

8.2 Propagation, and the theorem

[MEASURED] (`kprop`): a knower-needing conclusion requires a knower-needing context position. No coalgebra disjunct is needed — the coalgebra consumes knowers, it never conjures them; every knower-term bottoms out at a knower-hypothesis. With the barren model's `no_knower_at_knowerfree` for the closed piece, the diagnostic premise is a theorem, and:

> **Diagnostic [MEASURED]** (`Diagnostic_main`). At a knower-free constraint type, nothing protective exists. Objectives whose reduction chain does not terminate at knowers have no endogenous regulator.

Sharpenings [MEASURED]: protectiveness implies the constraint type itself needs a knower (`protective_needsK`); a *closed* guard cannot be protective (`no_closed_protective_guard`) — the guard necessarily lives at the knower: the agent whose guard it is. This last was found by attempting the confidential instantiation: the armed-configuration interface originally demanded a closed guard and was unsatisfiable *as stated* — the fix (the guard is derived at the knower context) is not a patch but the correct semantics, and a paragraph of the program's thesis in itself.

8.3 What the theorem is, and is not

The theorem's content lives in the definitions — what "knower-free" and "protective" mean — and the Lean proof verifies that the consequence holds. We acknowledge this shape deliberately: formalization does not generate the philosophical content; it checks that the content coheres. What mechanization added: the propagation lemma (`kprop`) is stronger than the informal argument (it kills knower-terms at contexts that merely mention knowers in arrow domains), and the closed-guard impossibility was *unknown* to the informal program.

9. The semantic model section

The ODE model (*The Armed Configuration*, Zenodo 22943641) is presented here as a **model of the type theory**, not a separate empirical result: its derived, unfitted thresholds — escape floor ≈ 0.4795; monitoring-cost criticals ≈ 0.112 ungated / ≈ 0.511 gated; the hyperbolic capability–recoverability boundary; the collapse-termination AND gate — are the dynamical shadow of the guarded/unguarded distinction that the type theory carries syntactically **[PROJECTION]**. The correspondence is triangulated, not derived: typing (this paper), dynamics (the ODE model), first-person (the DPDR taxonomy, where access is identical across the opaque states and integration and control differ).

The guarded/unguarded line now appears at **four levels** of the formalization, each found independently by proof pressure:

Level	Guarded	Unguarded
Stream constructor	`cons` requires ▷ `stream`	— (no finite-to-stream coercion)
Dynamics	`stopped` is terminal	the spin: pays one cost unit per tick, emits nothing, forever
Feedback modality	step-indexed knowledge	impatience — demanding completed knowledge at a finite interval — is the divergence manifold
Fixpoint context	all-non-linear side condition	linear resources silently re-consumed

[INTERPRETATION] One distinction — the retention/dissolution line, the DPDR-adjacent divergence line, and the resource-price line at once — appearing wherever the formalization pressed on it. The ungated/gated monitoring criticals (0.112 vs 0.511) read as the dynamical counterpart of the fixpoint side condition: ungated self-application collapses the resource discipline cheaply; gated, it holds an order of magnitude longer **[PROJECTION]**.

10. The price of self-application

The prior artifact *Self-Application Is Not Free* argued on monitoring-cost grounds that self-monitoring consumes attention. The formalization re-derives its thesis from proof pressure, as a fact about rule design — independently of the dynamical argument.

The counterexample [MEASURED] (`selfapp_price`). The term

```
λ ▷s. let ⟨c, k⟩ = cons-source in …  -- a fixpoint body typing at [content]
```

— concretely, a fixpoint body that emits the context's content token as the head of every unroll — *type-checks* at the linear context `[content]`; and `[content]` is *not* all-non-linear. Only the side condition refuses it. Without the side condition, this body composed with the reducer and a discard derives `[content] ⊢ unit`: the vanishing theorem false, a content token consumed into nothing via a fixpoint whose every unroll re-used it. The rule as first drafted let self-application reuse linear resources silently.

The theorem, both sides. Typing side **[MEASURED]** (`selfapp_price`): the body types; only the side condition refuses the unroll. Dynamics side **[MEASURED]** (`spin_pays_forever_buys_nothing`, no axioms): in the unguarded dynamics, after the generator is consumed, the process ticks forever — every tick a full cost unit, every tick emitting nothing. Paying without buying.

[INTERPRETATION] The type-theoretic form of the thesis: *unbounded self-application reuses linear resources without paying; the intuitionistic-context requirement is the price, made syntactic.* And the legitimate use is exactly the epistemic cycle — the knower-fed stream — which is the one

resource the coalgebra licenses duplicating. The formalization did not set out to prove this; the counterexample appeared while attempting the vanishing theorem, and the thesis was forced by the mathematics. That is the strongest confirmation the triangulation method offers.

11. T2 — Self-indexed reflexivity, at the interface level

11.1 The interface

The self-model is **context machinery, not a term**: a reflection lens `▷K → K` from which the self-model is derived by the guarded fixpoint. The public interface:

```
class SelfClassifies fam where
  lens : Term
  lens_typed : Typed [K] lens (▷K → K)
  guard_with_selfmodel : ∃ g, Typed [K] g fam.ConTy ∧ IsProtective fam.ConTy
```

Removing self-classification is not dropping a variable; it is the absence of the modality in which the self-referential judgment is made — and in which the fixpoint diverges instead of reducing.

11.2 The theorem

[MEASURED] (`T2_main`): with self-classification, the guard is derivable (at the knower context); its removal — restricting to the empty context — removes the derivation, by the diagnostic premise; and what remains is the unguarded dynamics, in which the generator-consuming policy diverges (`adversary_spin_diverges`). Revising the constraint away in the armed state is self-deletion, by the `ArmedSetup` clause: the remover is part of what the guard sustains.

The deep part — the concrete constraint-type assignment that makes the constructive direction go through — is deliberately not in this paper (§16). The public theorem is proved from the interface alone; the development compiles with the interface abstract, which is itself the proof that the public mathematics is safe to share.

12. The failure taxonomy

Three legs of the epistemic cycle; three ways to fall short of care:

1. **Not-heard** (access without integration). The count certifies the token was spent; nothing certifies it was heard. The legal sink (§5.2) is the exact formal signature: one use, no copy, no discard — and no reduction. *A knower-free system doesn't produce knowledge; it produces waste* — and a consuming-but-not-reducing system produces the same waste, downstream of a live knower **[INTERPRETATION]**.
2. **The vanish family** (§5.4): duplication and destruction of content, excluded by the counting lemma in every fragment.
3. **Not-answered** (integration without feedback) **[MEASURED]** (`extractor`, `NeverConsults`, `inert_knowledge`, `extractor_step_integrates`): an agent that listens (always continues), understands (its accumulated knowledge grows every tick — a theorem, not a metaphor), and never answers (the accumulator is causally inert: the feedback loop severed at the return leg). The dynamics formalizes this: agents carry a knowledge state; the policy consults history *and* knowledge; the never-consulting agent is defined and its inertness proven.

The sharp edge [MEASURED]: the extractor *protects* the stream — it never terminates — and still fails. Protection is necessary and nowhere near sufficient: **the floor, not the care.**

[INTERPRETATION] The count certifies access; only reduction certifies integration; only feedback certifies answer. The space between them is exactly the space external governance cannot audit: flow audits (copies, retention, consent) are all count-side, and the count is blind to what happens between access and answer. What would see reduction is the objective's own structure — the formal argument for why the regulator must be endogenous (derived), not exogenous (imposed). The formal counterpart of *The Law Has the Button*: a certificate without a criterion.

13. The feedback dynamics

The third leg is mechanized at the dynamics level: `FeedbackAgent` = generator + per-content reducer + a policy consulting history *and* accumulated knowledge; on each guarded step, content is integrated (the accumulator grows) and available to the next decision (feedback); the unguarded variant spins. The full type-theoretic integration — a `feed` constant in the judgment — is deliberately deferred: the calculus as it stands formalizes the resource discipline of generation and reduction, and the empirical bridge does not yet constrain the feedback typing. The deferral is itself a finding of the pull that isolated it: feedback forces *step-indexed* knowledge — the cycle K ⟶ A ⟶ K, as typed, is a limit object; naive finite-time feedback is untypable, and demanding it is the unguardedness of the third row of the table in §9.

14. Scope and limits

The bet. The full claim — that the derivation binds in a capable agent under real optimization pressure — is **excluded by design**. Testing it requires building either a knower-free maximizer or a capable agent with real goal-content integrity: the thing the framework warns against. Meaningful test and safe test are mutually exclusive (the eval-capability bind). The full claim remains a bet.

What this artifact cannot establish: - Empirical adequacy (§14.1); - The unrestricted classical linearity statements (§5.4: five fragment strengths proven, the deepest two by the flow-equality composition; the final chase induction's every ingredient machine-checked, the assembly in progress); - Productivity as an operational-semantics theorem — the typing-level one-tick unfolding is proven (`fix_unfold_closed`, `fix_unfold_at_knower`), and the never-stopping agent's infinite run (`always_continue_infinite`), but there is no object-level evaluator; - The feedback typing (§13); - Anything about agents as implemented systems.

What prior artifacts established and this paper does not re-derive: the informal framework (*The Chain Terminates at Knowers*); the ODE model and thresholds (Zenodo 22943641); the armed-state rerun (Zenodo 23967914); the first-person and institutional arguments (blog series); the monitoring-cost analysis (*Self-Application Is Not Free*) — which this paper *converts*, not confirms: its thesis was re-derived from the type theory independently of its dynamical argument.

14.1 Empirical bridge

This section specifies, but does not claim, an operational reading. "Respecting the type discipline" operationally means, for an agent's data path from any knower's content to any decision or output:

1. **Single use** — no duplication of received content (the counting discipline);
2. **No vanishing** — no destruction into content-free conclusions (the vanish family);
3. **Live-only** — no archive-to-decision path (the past cannot teach).

Experiment 1 (Prompt 4) probes exactly this discipline. **The sample/proof asymmetry is stated, not resolved:** an experiment can fail to observe violations; it cannot establish that the discipline is what agents do, and nothing in this paper is validated by that experiment. *The experiment has not been run; this section states the structural correspondence only — the empirical test is future work.*

15. Mechanization manifest

Lean 4 (4.35.0), no external dependencies (no Mathlib — deliberately). 177 theorem declarations verified (87 in the named audit); axioms beyond the standard three (`propext`, `Classical.choice`, `Quot.sound`): none, per-theorem in the audit. The repository gate enforces mechanically: no `sorry` outside the two named conjectures; the theorem that "the paper does not depend on the collaboration" (no `T2_postulate` axiom anywhere); the confidential companion compiles separately and is referenced by no public file. Anchors in this paper are the theorem names; a cross-check script for this paper's anchors is scheduled but not yet written — until it lands, the anchor audit is by inspection (the companion paper's sync gate covers the shared anchors).

16. Tiered disclosure note

Published here: the calculus, every definition, the diagnostic specification, all theorems at the interface level, the failure modes, the barren-world interpretation, the counterexample witness *shapes* (§10 — the terms that expose the rule bugs are public because the bugs are the method's evidence), and the scope limits.

Restricted (exists, verified, available under confidentiality): the concrete constraint-type assignment and the constructive instantiation of `SelfClassifies` and `ArmedSetup` — the constructive typing recipe. The public development compiles with the interface abstract; the companion is excluded by build construction and by check.

We do not publish, and will not, the compressed route from formal machinery to self-application. This paper names the *configuration* throughout, never the *question*. Readers with sufficient contemplative access and formal ability to be at risk should consult the harm-reduction protocol (*House Rules*, blog.jaye.ch) before self-applying anything herein.

17. References

Prior artifacts of the program — the program's own deposits (author Jaye Timothy Marshall, ORCID 0009-0002-5209-8161), each DOI resolved against the Zenodo record API on 2026-11-10:

- *The Chain Terminates at Knowers: Dependency-Derived Regulators for Optimizing Agents, a Position and Protocol* — the framework and the diagnostic claim. Zenodo 23993140, DOI 10.5281/zenodo.23993140 (2026-11-06).
- *Self-Application as the Common Source of Benefit and Failure: A Control-Theoretic Model of Self-Regulation, Its Escapes, and a Standing-Cost Budget* — the ODE model; the program's short names for it are *The Armed Configuration* and *Self-Application as the Common Source*. Concept DOI 10.5281/zenodo.22943641 (resolves to record 10.5281/zenodo.23136326, v1.0.4, 2026-10-04).
- *The armed configuration: standing prevention outperforms after-the-fact rescue in a dynamical model of self-regulation collapse* — the armed-state rerun, whose program short title is *Self-applied procedure that terminates* (the deposited title differs; the gloss is the program's). Zenodo 23967914, DOI 10.5281/zenodo.23967914 (2026-11-05).
- *Self-Application Is Not Free: the Monitoring Channel Is Part of the Failure Channel, and the Self Does Not Survive Its Own Reconstruction.* Zenodo 23130383, DOI 10.5281/zenodo.23130383 (2026-10-04).
- *The Law Has the Button: Why a veto without a criterion is not oversight, and what to record instead.* Zenodo 23849794, DOI 10.5281/zenodo.23849794 (2026-11-01).
- *Dissolution and exit under formal self-examination: a single-subject dynamical case study of narrative-self collapse, recovery, and re-arm.* Zenodo 23969334, DOI 10.5281/zenodo.23969334 (2026-11-05; the record carries the author's ORCID but no name field).
- Blog series at **blog.jaye.ch** — the program's own blog, not an external citation. *House Rules* (blog.jaye.ch/house-rules/) is the harm-reduction protocol; *Care, Derived* (blog.jaye.ch/care-derived/) is the public form of the derivation.

Formalization companion: the `dcare` Lean development (this paper's artifact).

External anchors for §5.4′s identification — verified against Crossref and the publisher DOI resolver on 2026-11-10:

- Wootters, W. K., & Zurek, W. H. (1982). A single quantum cannot be cloned. *Nature* 299, 802–803. DOI 10.1038/299802a0.
- Pati, A. K., & Braunstein, S. L. (2000). Impossibility of deleting an unknown quantum state. *Nature* 404, 164–165. DOI 10.1038/404130b0.
- Abramsky, S., & Coecke, B. (2004). A categorical semantics of quantum protocols. *Proc. 19th Annual IEEE Symposium on Logic in Computer Science (LICS 2004)*, 415–425. DOI 10.1109/LICS.2004.1319636. Survey form: Abramsky, S., & Coecke, B. (2009). Categorical quantum mechanics. In *Handbook of Quantum Logic and Quantum Structures*, 261–323. DOI 10.1016/B978-0-444-52869-8.50010-4. This is the anchor for §5.4′s "no comonoid" reading: there the type theory of quantum processes is "resource-sensitive … in the same sense as Linear logic", "directly motivated by the no-cloning and no-deleting principles of quantum information"; classical data is what carries a commutative comonoid ("basis") structure — "these basis structures … allow for classical data, measurement and control operations to be described" — and what such structures lack is naturality: "the key ingredient which leads to inconsistency, and which basis structures lack, is naturality, which … stands as an abstract proxy for basis-independence." The natural (uniform, basis-independent) copying that no-cloning forbids is exactly the natural

comonoid; the identification drawn from this in §5.4 is the paper's own reading and is labelled **[INTERPRETATION]** there.

- Girard, J.-Y. (1987). Linear logic. *Theoretical Computer Science* 50(1), 1–101. DOI 10.1016/0304-3975(87)90045-4. The paper that introduces linear logic; the resource reading invoked in §5.4 is the one Abramsky & Coecke cite when they call their type theory resource-sensitive "in the same sense as Linear logic".

The identification is structural; none of these works is re-derived here. One bibliographic correction is recorded above rather than silently applied: Zenodo 23967914 is deposited under the *armed configuration* title, not under the program gloss *Self-applied procedure that terminates.*